SDK ADOPTION · TECHNOLOGY BRIEFING

Enterprise Agentic AI: *one platform,* two teams.

§ 01 · THE PREMISE

AI agents are easy to demo. Operating them is the *hard* part.

What looks like a prototype problem...

A working demo is one prompt and a few lines of LangGraph. A working fleet is the moment every cross-cutting concern shows up at once.

Authentication. Authorisation. Audit. Observability. Retries. Timeouts. Schema versioning. Secret resolution. Token budgets. PII redaction. MCP

...is actually a platform problem.

We can solve those concerns once, in a shared SDK we audit carefully — or we can solve them eight times, in eight teams, with eight different bugs.

This deck argues for once. It defines the two teams who'll meet at that boundary, what each one owns, and what we ask of leadership to make the model

§ 02 · OUR APPROACH

One SDK. Two teams. *Zero drift.*

Three principles guide every binding choice, every interface, and every line of agent code we ship.

PRINCIPLE 01

Solve cross-cutting concerns once.

Auth, audit, observability, budgets, retries, schemas, secrets — all owned in the SDK. Shared, audited, tested centrally.

PRINCIPLE 02

Depend on interfaces, bind implementations.

Every load-bearing dependency sits behind an `I*` interface. Implementations are swappable; the contract is stable.

PRINCIPLE 03

Absorb library churn at the SDK boundary.

When LangGraph, LiteLLM, or Pydantic move, the SDK migrates internally — once. Agents pin only the SDK.

§ 03 · THE TWO TEAMS

Two teams meet at the *SDK boundary.*

Adopting the SDK requires a clear split of responsibility. Each team does what they do best; neither has to do the other's work.

GROUP A · MAINTAINS THE SDK

Platform / Tech AI Team

Architects · Software Engineers · DevOps · Production Support

Owns the runtime. Implements cross-cutting concerns once, with strong tests, clear contracts, and structured error semantics. Provides safe defaults so a careless agent can't ship an unsafe configuration.

GROUP B · BUILDS ON THE SDK

Business AI Team

Agent Engineers · MCP Tool Authors · RAG Engineers · Domain Experts

Owns the agents. Translates business problems into prompts, tools, schemas, and evaluations. Picks models, tunes them, measures them against business outcomes. Treats the SDK as a stable substrate.

§ 04 · WHAT THE PLATFORM TEAM OWNS

Cross-cutting concerns, solved *once* for everyone.

01 **Authentication & Authorisation**
JWT verification + OPA policy at every tool and HTTP boundary; fail-closed by default.

02 **Audit pipeline**
Structured AuditRecord events; redactor chain; fan-out to file, Sentry, Datadog.

03 **Observability**
OpenTelemetry traces + LangFuse generations + structlog with task-isolated context.

04 **Budget enforcement**
Per-tenant and per-(tenant, agent) limits on tokens and USD, with daily reset.

05 **Retry & timeout**
Tenacity-backed policies with consistent jitter, status-aware backoff, cancellation.

06 **Schema versioning**
Versioned Pydantic schemas at every tool boundary; ingress + egress validation.

07 **Secret management**
Opaque references resolved through ISecretManager — env, AWS, Vault, never inline.

08 **MCP integration**
Connection pool with idle TTL and health; late-binding-safe tool resolution.

09 **Memory & compaction**
Token-aware summarisation with timeout guard; preserves Essential

10 **DI & test seam**
A real DI container is the only seam; fakes provided for every interface.

§ 05 · WHAT THE BUSINESS AI TEAM OWNS

The work only *you* can do.

When the SDK takes the infrastructure off your plate, this is what you spend your time on instead — the parts agentic AI actually exists to deliver.

→ Prompt engineering & evaluation
System prompts, few-shot examples, eval harnesses against business outcomes.

→ Tool design
What should be a tool, at what granularity, with what policy boundary.

→ RAG pipelines
Retrieval quality, chunking, re-ranking — the parts unique to your domain knowledge.

→ Agent topology
Single agent vs hand-off vs supervisor; when to add an MCP server vs a tool.

→ Domain modelling
The Pydantic schemas your tools take in and return — the language of the business.

→ Model selection & tuning
Which LLM for which agent; temperature, max-tokens, fine-tuning where it pays off.

→ Business-outcome metrics
Resolution rate, escalation rate, cost-per-task — not just latency & tokens.

→ Tests against fakes
Tests that catch business-logic regressions, run in 0.04s, no external services.

§ 06 · WHAT WE AVOID

What happens *without* a shared SDK.

02:14 · TUESDAY

The 2 a.m. incident

Billing-agent latency p99 spikes to 8s. Logs in three formats from three teams. No shared trace_id. Mean time to root-cause: hours.

$34,000 · OVERNIGHT

The midnight burn

A tool retries on every 500. By morning the agent has burned tens of thousands in inference credits. Your CFO has questions. No central budget enforcement.

Q3 · AUDIT WINDOW

The compliance audit

Regulator asks for a complete trail of AI-driven decisions. Logs are scattered across four sources with four schemas. Some events were never written.

MAJOR VERSION BUMP

The library upgrade

LangGraph ships breaking changes. Every team migrates independently, on their own schedule, with their own bugs. The migration drags on for two quarters.

§ 07 · ARCHITECTURE AT A GLANCE

A layer cake, *not* a maze.

BUSINESS AI TEAM — YOU WRITE THIS

Business AI Agents

support · triage · KB-search · approvals · onboarding · domain RAGs

↓

PLATFORM TEAM — PLATFORM TEAM OWNS

ai-core-sdk

DI runtime · async · typed · observable · policy-aware

AUTH · OPA	AUDIT · REDACT	OTel · LangFuse	BUDGET	SCHEMA · VERSION	SECRETS · ISecretManager
RETRY · TIMEOUT	MCP POOL	MEMORY · COMPACT	DI · FAKES	HEALTH · CONFIG	CLI · SCAFFOLDING

↓

EXTERNAL SYSTEMS — VENDORS & INFRA

Postgres · OPA · LLM providers · MCP servers · OTel · LangFuse · Sentry/Datadog · Secret stores

§ 08 · WHO OWNS WHAT

Most adoption friction is *ambiguous* ownership.

CONCERN	PLATFORM TEAM	BUSINESS AI TEAM
Authorisation	JWT verifier, OPA evaluator, fail-closed defaults	Choosing `opa_path` per tool
Tools	@tool decorator, schema registry, invoker pipeline	Tool inputs/outputs & handler logic
Observability	OTel + LangFuse wiring, baggage, log shape	Calling `bind_context()` with the right ids
Budget	BudgetService, daily reset, alerting	Configuring per-(tenant, agent) limits
LLM calls	LiteLLM client, retry, timeout, token counting	Picking the model, writing the prompt
Memory	MemoryManager, compaction, token-budget guard	`essential_entity_keys` for the use case
MCP integration	Connection pool, lifecycle, late-binding-safe resolver	Declaring an `MCPServerSpec`
Testing	DI fakes for every interface, conftest fixtures	Writing tests against fakes, not real services
Upgrades	LangGraph / LiteLLM / OPA migrations absorbed	Following the SDK CHANGELOG on minor

§ 09 · NO VENDOR LOCK-IN BY DESIGN

Every load-bearing dependency sits behind an *interface.*

Replace the implementation, keep the agents. Most swaps are a single DI binding change.

`ILLMClient`	SWAP · LOW LiteLLM today; OpenRouter, Portkey, vendor SDK tomorrow.
`IObservabilityProvider`	SWAP · LOW OTel + LangFuse today; Datadog APM, Honeycomb, Helicone tomorrow.
`IStorageProvider`	SWAP · LOW SQLAlchemy + asyncpg; alternative drivers via URL.
`ISecretManager`	SWAP · LOW Env, AWS, Vault — bind the backend you need.
`IPolicyEvaluator`	SWAP · LOW OPA today; AWS Cedar or Casbin tomorrow.
`JWTVerifier`	SWAP · LOW PyJWT today; authlib, jwcrypto, upstream gateway tomorrow.
`IMCPConnectionFactory`	SWAP · LOW fastmcp today; alternative MCP clients drop in.
`IAuditSink`	SWAP · LOW File, Sentry, Datadog — fan-out to whichever sinks you run.

§ 10 · EVOLUTION OVER TIME

The SDK is the *dampening* layer.

LITELLM 1.32 → 1.40	Streaming response shape changed. Diff lived in *LiteLLMClient._normalize_response*. Agent code: zero changes.
LANGGRAPH 0.0.50 → 0.0.55	Reducer signature broadened with *REMOVE_ALL_MESSAGES*. SDK adopted internally; compaction node updated. Agent code: zero changes.
FUTURE · LANGGRAPH 1.0	Anticipated graph-API stabilisation. SDK absorbs the migration once. Agents using BaseAgent + @tool: expected zero changes.
FUTURE · PYDANTIC V3	SDK pins to v2 until v3 is stable, then migrates schemas centrally with explicit CHANGELOG entries. Idiomatic agent code: minimal touch-up.

The promise: The SDK pins major versions; agents pin only the SDK. Breaking changes in dependencies are absorbed inside the SDK. Deprecations get a release-cycle of warning before removal.

§ 11 · ADOPTION PATH

A short, low-ceremony rollout.

No stop-the-world rewrite. No big-bang. Existing agents migrate as they're touched; new agents start on the SDK from day one.

PHASE 01

Week 1 · Pilot

Pick one in-flight agent. Move it onto BaseAgent + @tool. Capture friction in a shared doc.

PHASE 02

Week 2 – 3 · Close gaps

Platform team addresses the friction list. Business team writes the second agent — noticeably faster.

PHASE 03

Week 4 · Production gates

Set the bar: any agent shipping to a regulated environment runs on the SDK. Audit + observability shape becomes the deployment gate.

PHASE 04

Quarter 2 · Migrate the rest

Existing agents migrate one by one as they touch them. New agents start on the SDK from day one.

§ 12 · SUCCESS LOOKS LIKE

What we'll *measure.*

Concrete, agreed targets. If we hit these, the model is working. If we don't, we revisit.

Time to ship a new production agent

days, *not quarters*

Mean time to root-cause an incident

minutes, *not hours*

Compliance audit prep — query to answer

hours, *not weeks*

Coordinated dependency migrations across teams

zero — *SDK absorbs them*

Unit-test coverage of agents (no live services)

≥ 80% *with fakes only*

Budget-blowout incidents per quarter

zero — *enforced centrally*

§ 13 · WHAT WE'RE ASKING FOR

Three commitments from *leadership.*

01

Recognise the SDK as a first-class product.

Not invisible plumbing. Versioning, a CHANGELOG, a clear support model, and time on the platform team's roadmap. Without this, the SDK becomes someone's side project and the model collapses.

02

Mandate the SDK for production deployments after pilot validation.

Once the pilot agent ships, the deployment gate is "runs on the SDK". This prevents fragmentation before it happens — much cheaper than fixing it after.

03

Trust the platform/business contract.

Don't ask the platform team to ship business outcomes. Don't ask the business AI team to maintain platform infrastructure. Each team is set up for what they do best — let them.

Build it *once.*
Build it *well.*
Build it *together.*

The SDK is the contract between two teams who otherwise wouldn't agree. With it, both teams ship faster, safer, and with less drift than either could alone.